Homework 4

NOTE: USED 3 LATE DAYS

Matthew Wilkinson

mmw071@uark.edu

100434077

1. Describe the purpose of the homework and what you did / what you implemented.
2. Describe your approach and explain how you solved the problem (the key ideas for implementing each required function).
3. Discuss any bugs or problems you encountered and how you fixed them.
4. Include screenshots or images of your results.

Purpose

The purpose of the homework was to practice implementing several algorithms. These included sorting algorithms, graph algorithms, and a balanced tree. The goal was to better understand how these algorithms work and how they are written.

This homework requires implementation of the following:

- Heap Sort
- Merge Sort
- Topological Sort (Kahn's)
- Prim's Algorithm for MST
- Kruskal's Algorithm for MST
- Dijkstra's Algorithm
- Red-Black Tree

Approach

Heap Sort: I first build a max heap from the array. Then I repeatedly swapped the root with the last element and called heapify to maintain the heap. This continues until the array is sorted.

Merge Sort: I used recursive approach from lecture. The array is divided into two halves until each part has one element. Then the merge function combines the sorted halves together back in order.

Topological Sort: I used Kahn's Algorithm as instructed. I first calculated the in degree of every vertex. Then I added all the vertices with in degree equal to zero into the queue. I removed the vertices one by one, updating the in degree value as needed, until all vertices were processed.

Prim's Algorithm: I built the MST by starting from a vertex and always choosing the smallest edge that connects a visited vertex to an unvisited vertex.

Kruskal's Algorithm: I first sorted all the edges by weight using Merge Sort. Then I used the Disjoint Set structure to check if adding an edge would create a cycle. If it didn't, the edge was added to the MST.

Dijkstra's Algorithm: I used a priority queue to always process the vertex with the smallest distance. I updated the distances for neighboring vertices and stored the previous vertex so the final path can be reconstructed.

Red-Black Tree: I implemented the insert function and the balancing logic. After inserting the node as red, the program checks for violations given the red-black tree rules. If a violation occurs, the tree is fixed using recoloring and rotations.

Bugs/Problems

One problem I had was with MakeFile when trying to compile the program using mingw32-make. The commands inside the MakeFile used windows style commands like "if not exist", but mingw32 make was using a different shell, I guess. This caused syntax errors when compiling. I fixed this by replacing the commands with shell compatible commands.

As usual with me, OpenCV continues to be a pain, so I didn't run it on my machine. Besides that, everything else appears to work correctly on my machine.

RESULTS SECTION ON NEXT PAGE

Results

Terminal of Mingw64 of Final Results:

```
/c/Users/mww61/gitRepos/algor/algorithmsclass/hw4

mww61@Matthew MINGW64 /c/Users/mww61/gitRepos/algor/algorithmsclass/hw4
$ mingw32-make
mkdir -p bin
g++ -I./include/ -std=c++14  -DKRUSKAL -DDIJKSTRA -DPRIM -DINTEGRATED src/data_structures/disjoint_
et.cpp src/graph.cpp src/main.cpp src/sort/msort.cpp src/sort/hsort.cpp src/algorithms/topologicals
rt.cpp src/algorithms/rbtree.cpp src/algorithms/kruskal.cpp src/algorithms/dijkstra.cpp src/algorit
ms/prim.cpp  -o bin/integrated.exe
./bin/integrated.exe
Perform unit test on heap sort
Original array: 41 17 34 0 19 24 28 8 12 14 5 45 31 27 11 41 45 42 27 36
Sorted array: 0 5 8 11 12 14 17 19 24 27 27 28 31 34 36 41 41 42 45 45
Your sorting implementation is correct

Perform unit test on Merge sort
Original array: 41 4 2 3 42 32 21 16 18 45 47 26 21 38 19 12 17 49 35 44
Sorted array: 2 3 4 12 16 17 18 19 21 21 26 32 35 38 41 42 44 45 47 49
Your sorting implementation is correct


Perform unit test on Topological sort
Topological Sort: 4 5 0 2 3 1
Topological Sort implementation is correct.
Perform unit test of the Prim's algorithm
Total Cost of Minimum Spanning Tree: 16

Total Cost of Minimum Spanning Tree: 9315
You have to use OpenCV to visualize your map

Perform unit test of the Kruskal's algorithm
Total Cost of Minimum Spanning Tree: 16

Total Cost of Minimum Spanning Tree: 9315
You have to use OpenCV to visualize your map

Perform unit test of the Dijkstra algorithm
Path from 0 to 5: 0 -> 1 -> 2 -> 4 -> 5

Path from SAEF to GRAD: SAEF -> BUCH -> LCAF -> GLAD2 -> PDTF -> SINF -> LMK111 -> WAHR -> SDPG2 ->
ARKU2 -> POST -> FPAC -> WATR -> ARSAGAS -> CORD1 -> PTSC -> MEMH2 -> ARMY -> GRAD
You have to use OpenCV to visualize your map road

Perform unit test of the Red-Black Tree

==========================================================
--- STARTING INTEGRATED ALGORITHM STACK DEMO ---
==========================================================
[Step 1] RB-Tree: Coordinates stored for Buildings 0, 1, and 2.
 -> Found Building 0 at (100, 100)
 -> Found Building 1 at (200, 300)
 -> Found Building 2 at (400, 150)

[Step 2] Graph: Edges created with Euclidean weights:
 -> Edge (0,1) weight: 223
 -> Edge (1,2) weight: 250
 -> Edge (0,2) weight: 304

[Step 3] Kruskal: Constructing Minimum Spanning Tree...
 -> Added Edge (0,1) w:223
 -> Added Edge (1,2) w:250
Total MST Cost: 473

[Step 4] Dijkstra: Calculating shortest path from 0 to 2...
 -> Shortest Path: 0 -> 2

[Step 5] HeapSort: Sorting traffic congestion scores...
 -> Original: 85 12 44 98 30
 -> Sorted:   12 30 44 85 98

==========================================================
--- INTEGRATED DEMO COMPLETE ---
==========================================================

mww61@Matthew MINGW64 /c/Users/mww61/gitRepos/algor/algorithmsclass/hw4
```

Run Time Complexity

Heap Sort runs in O(n log n) time. Building the heap takes O(n), and each removal from the heap takes O(log n), which happens n times.

Merge Sort runs in O(n log n) time. The array is repeatedly split into halves and then merged back together. There are log n levels of recursion and each level processes n elements.

Topological Sort runs in O(V + E) time. Each vertex and each edge is processed once while calculating in degrees and removing vertices from the queue.

Prim's Algorithm runs in O(E log V) time. The algorithm repeatedly selects the smallest edge connecting a visited vertex to an unvisited vertex.

Kruskal's Algorithm runs in O(E log E) time. The edges are first sorted by weight, and then each edge is checked using the disjoint set structure to avoid cycles.

Dijkstra's Algorithm runs in O(E log V) when using a priority queue. Each vertex is processed based on the smallest known distance and edges are relaxed.

Red-Black Tree insertion runs in O(log n) time. The tree remains balanced after each insertion using rotations and recoloring, so its height stays logarithmic.